

TP03 - Compte rendu - BASH \$

Année 2025 – 2026

Exercice : paramètres

Objectif : Créer un script analyse.sh qui affiche le nombre de paramètres, le nom du script, le troisième paramètre et la liste des paramètres.

Script :

```
#!/bin/bash
echo "Bonjour, vous avez rentré $# paramètres."
echo "Le nom du script est $0"
echo "Le 3eme paramètre est $3"
echo "Voici la liste des paramètres : $@"
```

Explication :

`$#` → nombre de paramètres
`$0` → nom du script
`$3` → troisième paramètre
`$@` → liste complète des paramètres

Exemple :

```
./analyse.sh un deux trois
```

Résultat :

```
Bonjour, vous avez rentré 3 paramètres.
Le nom du script est ./analyse.sh
Le 3eme paramètre est trois
Voici la liste des paramètres : un deux trois
```

Exercice : vérification du nombre de paramètres

Objectif : Créer un script concat.sh qui vérifie qu'il y a exactement 2 paramètres et concatène les deux mots.

Script :

```
#!/bin/bash
if [ $# -ne 2 ]; then
    echo "Erreur : vous devez entrer exactement 2
paramètres"
    exit 1
fi
```

```
CONCAT="$1$2"
echo "Résultat : $CONCAT"
```

Explication :

Ce script vérifie que l'utilisateur entre exactement deux paramètres. L'opérateur -ne signifie 'différent de'. Si la condition n'est pas respectée, le script quitte avec exit 1. Les deux mots sont concaténés dans la variable CONCAT en juxtaposant \$1 et \$2.

Exemple :

```
./concat.sh bon jour
Résultat : bonjour
```

Exercice : argument type et droits

Objectif : Identifier le type d'un fichier passé en paramètre et afficher les droits d'accès de l'utilisateur courant.

Script :

```
FICHIER=$1
if [ -d "$FICHIER" ]; then
    echo "Le fichier $FICHIER est un répertoire"
elif [ -f "$FICHIER" ]; then
    if [ -s "$FICHIER" ]; then
        echo "Le fichier $FICHIER est un fichier ordinaire
non vide"
    else
        echo "Le fichier $FICHIER est un fichier ordinaire
vide"
    fi
else
    echo "Le fichier $FICHIER est d'un type spécial"
fi
USER=$(whoami)
echo -n "\"$FICHIER\" est accessible par $USER en "
[[ -r "$FICHIER" ]] && echo -n "lecture " || echo -n ""
[[ -w "$FICHIER" ]] && echo -n "écriture " || echo -n ""
[[ -x "$FICHIER" ]] && echo -n "exécution" || echo -n ""
echo
```

Explication :

- d : vérifie si c'est un répertoire
- f : vérifie si c'est un fichier ordinaire

-s : vérifie si le fichier n'est pas vide
-r / -w / -x : droits en lecture, écriture, exécution
whoami : retourne l'utilisateur courant

Exemple de résultat :

```
Le fichier /etc est un répertoire
"/etc" est accessible par root en lecture écriture
exécution
```

Exercice : afficher le contenu d'un répertoire

Objectif : Lister séparément les fichiers et sous-répertoires d'un dossier passé en argument.

Script :

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Erreur : vous devez fournir exactement 1
    répertoire"
    exit 1
fi
REP="$1"
if [ ! -d "$REP" ]; then
    echo "$REP n'est pas un répertoire"
    exit 1
fi
echo "##### fichiers dans $REP/"
for f in "$REP"/*; do
    if [ -f "$f" ]; then
        echo "$f"
    fi
done
echo "##### repertoires dans $REP/"
for d in "$REP"/*; do
    if [ -d "$d" ]; then
        echo "$d"
    fi
done
```

Explication :

La boucle for f in "\$REP"/* parcourt tous les éléments du dossier. On fait deux passes séparées pour distinguer fichiers (-f) et répertoires (-d).

Exemple :

```
./listdir.sh /boot
##### fichiers dans /boot/
/boot/config-3.16.0-4-amd64
/boot/initrd.img-3.16.0-4-amd64
##### repertoires dans /boot/
/boot/grub
```

Exercice : lister les utilisateurs

Problème avec `for user in $(cat /etc/passwd)` : le shell découpe sur les espaces et non sur les lignes, ce qui casse les données.

Solution avec cut :

```
#!/bin/bash
cut -d: -f1,3 /etc/passwd | while IFS=: read login uid
do
    if [ "$uid" -gt 100 ]
    then
        echo "$login"
    fi
done
```

Solution avec awk :

```
#!/bin/bash
awk -F: '$3 > 100 {print $1}' /etc/passwd
```

Explication :

Le fichier `/etc/passwd` a la structure : `login:x:UID:GID:commentaire:home:shell`

`cut -d: -f1,3` : extrait le login (colonne 1) et l'UID (colonne 3)

`IFS=:` permet à `read` de découper selon les deux-points

`awk -F:` : solution plus concise qui combine filtrage et affichage en une ligne

Exercice : vérifier si un utilisateur existe

Script :

```
#!/bin/bash
if [ $# -ne 1 ]
then
    echo "Usage : $0 login"
    exit 1
```

```

fi
login=$1
uid=$(grep "^$login:" /etc/passwd | cut -d: -f3)
if [ -n "$uid" ]
then
    echo "$uid"
fi

```

Explication :

grep "^\$login:" cherche une ligne commençant exactement par le login suivi de :. cut -d: -f3 récupère l'UID. Le test -n vérifie que la variable n'est pas vide avant l'affichage.

Exercice : création utilisateur

Script :

```

#!/bin/bash
if [ "$USER" != "root" ]; then
    echo "Vous devez être root pour créer un utilisateur"
    exit 1
fi
read -p "Login : " LOGIN
read -p "Nom : " NOM
read -p "Prenom : " PRENOM
read -p "UID : " UID
read -p "GID : " GID
read -p "Commentaire : " COMMENTAIRE
if grep -q "^$LOGIN:" /etc/passwd; then
    echo "L'utilisateur existe déjà"
    exit 1
fi
if [ -d "/home/$LOGIN" ]; then
    echo "Le répertoire /home/$LOGIN existe déjà"
    exit 1
fi
useradd -m -u "$UID" -g "$GID" -c "$PRENOM $NOM,
$COMMENTAIRE" "$LOGIN"

```

Explication :

Le script vérifie d'abord que l'exécutant est root. Il collecte les infos via read -p. Deux vérifications sont faites : existence du login dans /etc/passwd et existence du dossier

home. La commande `useradd` crée le compte avec `-m` (créer `/home`), `-u` (UID), `-g` (GID), `-c` (commentaire).

Commandes utilisées :

```
echo -n "Entrer votre nom: "  
read nom  
echo "Votre nom est $nom"
```

La commande `read` permet de lire une valeur saisie au clavier et de la stocker dans une variable.

Exercice : visualiser les fichiers texte d'un répertoire

Commande `file` : identifie le type d'un fichier en analysant son contenu.

Commande `more` : affiche le contenu d'un fichier page par page (q pour quitter, Espace pour avancer, b pour reculer, /mot pour chercher).

Script :

```
#!/bin/bash  
if [ $# -ne 1 ]; then  
    echo "Erreur : vous devez fournir un répertoire"  
    exit 1  
fi  
REP="$1"  
for fichier in "$REP"/*; do  
    TYPE=$(file "$fichier")  
    if [[ $TYPE == *text* ]]; then  
        echo -n "Voulez vous visualiser le fichier $fichier  
? (o/n) : "  
        read REPONSE  
        if [ "$REPONSE" = "o" ]; then  
            more "$fichier"  
        fi  
    fi  
done
```

Explication :

La syntaxe `[[ $TYPE == *text* ]]` utilise le pattern matching Bash pour détecter si la sortie de `file` contient le mot `text`. Si oui, l'utilisateur confirme l'affichage avec `more`.

Exercice : Appréciation

Script :

```
#!/bin/bash
```

```

while true
do
    echo -n "Entrer une note (ou q pour quitter) : "
    read NOTE
    if [ "$NOTE" = "q" ]; then
        exit
    fi
    if [ "$NOTE" -ge 16 ] && [ "$NOTE" -le 20 ]; then
        echo "très bien"
    elif [ "$NOTE" -ge 14 ]; then
        echo "bien"
    elif [ "$NOTE" -ge 12 ]; then
        echo "assez bien"
    elif [ "$NOTE" -ge 10 ]; then
        echo "moyen"
    else
        echo "insuffisant"
    fi
done

```

Explication :

La boucle while true tourne indéfiniment jusqu'à la saisie de q. Les opérateurs -ge (supérieur ou égal) et -le (inférieur ou égal) comparent des valeurs numériques. Les appréciations sont : 16-20 très bien, 14-15 bien, 12-13 assez bien, 10-11 moyen, moins de 10 insuffisant.

Sources

1. GNU Bash Manual — <https://www.gnu.org/software/bash/manual/>
2. Linux manual pages — man bash, man read, man file, man more
3. The Linux Documentation Project — <https://tldp.org>